

GO PROGRAMMING MASTER COURSE

Week 7, Day 1

Structs, JSON tags, and in-memory students —
create and list via Postman.

Umesh Indrajith · BSc Eng (Hons), University of Moratuwa
Senior Software Engineer / Lecturer

SINCE
LAST WEEK

Handlers extracted. Now give them data shapes.

- Health + hello still 200. `internal/handlers` is real.
- Cold-call: method vs function? Why pointer receiver on handlers?
- Today we add **structs** and an in-memory student store.
- Still no database. Still no JWT. Product grows in memory first.

TODAY **If Day 1 of week 7 works, you can**

- Define a struct with exported fields and composite literals.
- Round-trip JSON with tags like ``json:"first_name"``.
- Explain a tiny interface (error) and “accept interfaces, return structs”.
- Store students in a `map[int]Student` (or slice).
- Ship `POST /api/students`, `GET /api/students`, `GET /api/students/:id`.

**SOFTWARE
ENGINEERING**

A model is a shared language

JSON field names are a contract with Postman and later the React client. Tags make Go names and wire names both sane.

- SMS fields: first/last name, email, phone, course, date of birth.
- In-memory now → same shapes against Postgres in week 14.
- Invalid JSON → **400**, not a panic.

STRUCTS

Named fields, one type

```
type Student struct {  
    ID          int  
    FirstName   string  
    LastName    string  
    Email       string  
    Course      string  
}
```

```
s := Student{FirstName: "Ada", LastName: "Lovelace",  
             Email: "ada@lovelace.edu", Course: "Go Programming"}  
fmt.Println(s.FirstName)
```

- Capital field = exported (visible outside the package / in JSON by default).
- Composite literal: set the fields you care about; others get zero values.

JSON
TAGS

Wire names that match the API

```
type Student struct {  
    ID          int    `json:"ID"`  
    FirstName   string `json:"first_name"`  
    LastName    string `json:"last_name"`  
    Email       string `json:"email"`  
    Course      string `json:"course"`  
}
```

Without tags, JSON would say FirstName. We want first_name.

Match this repo's model tags. Ignore DB tags until week 14.

METHODS
ON
STRUCTS

Behaviour next to data

```
func (s Student) FullName() string {  
    return s.FirstName + " " + s.LastName  
}  
// or reuse handlers.FullName(s)
```

- Value receiver is fine for read-only helpers.
- Store / handler that mutates the map
→ pointer receiver on the handler type.

EMBEDDING
• LIGHT

Composition, not inheritance

```
type Timestamps struct {  
    CreatedAt time.Time `json:"created_at"`  
}  
  
type Student struct {  
    ID int `json:"id"`  
    Timestamps // embeds  
}
```

- Go has no class hierarchy. Embed when you want fields/methods promoted.

- Optional today — do not force embedding in the lab.

INTERFACES
• LIGHT

Small surfaces. error is an interface.

```
type error interface {  
    Error() string  
}  
// later: type StudentStore i
```

- **Accept interfaces, return structs** — depend on behaviour, hand back concrete values.

- Today you may still return concrete []Student. Plant the proverb.

IO.READER

The request body is a reader

- `io.Reader` — something you can read bytes from.
- `io.Writer` — something you can write bytes to.
- Gin's `c.Request.Body` is a reader; `ShouldBindJSON` reads it into your struct.
- That is why interfaces matter: one decode function works for files, buffers, and HTTP bodies.

IN-
MEMORY
STORE

map[int]Student is enough this week

```
var (  
    mu      sync.Mutex // optional  
    students = map[int]models.Student  
    nextID   = 1  
)
```

- Create assigns nextID, stores, increments.
- List ranges the map (order not promised — sort IDs if tests need order).
- Get by id: comma-ok. Missing → 404.

SMS
MILESTONE

Three routes today

```
POST /api/students
GET  /api/students
GET  /api/students/:id
```

No JWT. No Postgres.

```
→ 201
→ 200
→ 200
```

Request body uses first_name, last_name, email, course, ...

Bad JSON / missing required fields → **400** with {"error": "..."}.

LIVE
DEMO

MODELS

Add internal/models/student.go

```
package models
type Student struct {
    ID          int    `json:"id"`
    FirstName   string `json:"first_name"`
    LastName    string `json:"last_name"`
    Email       string `json:"email"`
    Phone       string `json:"phone"`
    Course      string `json:"course"`
}
```

LIVE
DEMO

CREATE

POST /api/students

```
func (h *StudentHandler) Create(c *gin.Context) {  
    var req models.Student // or CreateStudentRequ  
    if err := c.ShouldBindJSON(&req); err != nil {  
        c.JSON(400, gin.H{"error": "invalid json"})  
        return  
    }  
    // assign id, store in map, c.JSON(201, saved)  
}
```

LIVE
DEMO

LIST +
GET

GET collection and by id

```
r.POST("/api/students", student.Create)
r.GET("/api/students", student.List)
r.GET("/api/students/:id", student.GetByID)
```

```
// GetByID: id, err := strconv.Atoi(c.Param("id"))
// s, ok := students[id]; if !ok { 404 }
```

POSTMAN

Collection grows

- Folder Week 7: Create student, List students, Get by id.
- Create: Body → raw → JSON with first_name etc.
- Save an example 201 response. Homework will add Update / Delete.

```
{  
  "first_name": "Ada",  
  "last_name": "Lovela",  
  "email": "ada@example",  
  "phone": "0770000000",  
  "course": "SE"  
}
```

VALIDATION

Reject junk with 400

- ShouldBindJSON fails → 400.
- Empty `first_name` / `email` → 400 with a clear error string.
- Do not panic. Do not return 500 for bad client input.
- Gin binding: "required" is fine if you introduce a request struct.

404 Missing id is not a server crash

```
s, ok := students[id]
if !ok {
    c.JSON(http.StatusNotFound, gin.H{"error": "id not found"})
    return
}
c.JSON(http.StatusOK, s)
```

- Comma-ok again — same idea as week 3 maps.

- Wrong id type (/api/students/abc) → 400.

LAB • REST OF
CLASS

Done when all of this is true

- `models.Student` with JSON tags exists
- Postman **POST** creates a student (201)
- Postman **GET** list shows it
- Get by id works; unknown id → 404
- Broken JSON → 400
- Health still green; commit / PR started

HOMEWORK

Update + delete

- PUT /api/students/:id — replace fields; missing → 404; bad JSON → 400.
- DELETE /api/students/:id — remove; missing → 404.
- Postman screenshots or collection export in the PR.
- Still no JWT / Postgres.

**NEXT
SESSION**

Week 8 — pointers, memory, repository layer

- Move the map into a StudentRepository.
- Handlers call the repo — HTTP must not own storage forever.
- Bring working create + list (and homework update/delete if done).

WORDS TO
KEEP

Say these correctly next week

Struct

Grouped fields under one type.

JSON tag

Renames fields on the wire.

Interface

Set of methods; error is one.

In-memory

Store in process RAM — lost on restart.

Where the new files live

```
backend/  
  cmd/server/main.go  
  internal/models/student.go  
  internal/handlers/  
    health.go  
    hello.go  
    student.go          # Create, List, GetByID (+ homework)  
    name.go
```

**HONEST
LIMITS**

What this week is not

- Not durable — restart wipes students.
- Not multi-user safe yet — races possible under load (week 12).
- Not authenticated — anyone can POST (week 15).
- That is intentional spiral teaching.

SAFETY

Still true with POST bodies

- Never log passwords (none today — good).
- Do not commit real emails of classmates without consent — use example.com.
- gofmt; no binaries; .env still ignored.

Questions